

```
-----
-----
ValueError                                Traceback (most recent call
last)
```

```
Cell In[20], line 64
```

```
    62 fig.suptitle(f'Fixed  $\omega_0$ = $\{\omega_0\}$ , varying  $\alpha$ ')
    63 plt.tight_layout()
--> 64 plt.savefig(f"Fixed  $w_0$ = $\{\omega_0\}$ , varying
alpha")
    65 plt.show()
    67 #
```

```
=====
=====
    68 # 2) Three figures: fixed  $\alpha$ ,  $\omega_0$  varies over
omega_vec
    69 #
```

```
=====
=====
File ~/miniconda3/envs/ecg-byte/lib/python3.10/site-
packages/matplotlib/pyplot.py:1228, in savefig(*args,
**kwargs)
    1225 fig = gcf()
    1226 # savefig default implementation has no return, so
mypy is unhappy
    1227 # presumably this is here because subclasses can
return?
-> 1228 res = fig.savefig(*args, **kwargs) # type:
ignore[func-returns-value]
    1229 fig.canvas.draw_idle() # Need this if
'transparent=True', to reset colors.
    1230 return res
```

```
File ~/miniconda3/envs/ecg-byte/lib/python3.10/site-
packages/matplotlib/figure.py:3395, in
Figure.savefig(self, fname, transparent, **kwargs)
    3393 for ax in self.axes:
    3394     _recursively_make_axes_transparent(stack,
ax)
```

```
-> 3395 self.canvas.print_figure(fname, **kwargs)
```

File ~/miniconda3/envs/ecg-byte/lib/python3.10/site-packages/matplotlib/backend_bases.py:2144, in FigureCanvasBase.print_figure(self, filename, dpi, facecolor, edgecolor, orientation, format, bbox_inches, pad_inches, bbox_extra_artists, backend, **kwargs)

```
2139     _api.warn_deprecated(
2140         "3.8", name="papertype='auto'",
addendum="Pass an explicit paper type, "
2141         "'figure', or omit the *papertype* argument
entirely.")
```

2143 # Remove the figure manager, if any, to avoid resizing the GUI widget.

```
-> 2144 with cbook._setattr_cm(self, manager=None), \
```

```
2145
self._switch_canvas_and_return_print_method(format,
backend) \
```

```
2146     as print_method, \
2147     cbook._setattr_cm(self.figure, dpi=dpi), \
2148     cbook._setattr_cm(self.figure.canvas,
_device_pixel_ratio=1), \
2149     cbook._setattr_cm(self.figure.canvas,
_is_saving=True), \
2150     ExitStack() as stack:
2152     for prop in ["facecolor", "edgecolor"]:
2153         color = locals()[prop]
```

File ~/miniconda3/envs/ecg-byte/lib/python3.10/contextlib.py:135, in _GeneratorContextManager.__enter__(self)

```
133 del self.args, self.kwds, self.func
134 try:
--> 135     return next(self.gen)
136 except StopIteration:
137     raise RuntimeError("generator didn't yield")
from None
```

File ~/miniconda3/envs/ecg-byte/lib/python3.10/site-packages/matplotlib/backend_bases.py:2040, in

```

FigureCanvasBase._switch_canvas_and_return_print_method(self, fmt, backend)
2038 canvas_class =
get_registered_canvas_class(fmt)
2039 if canvas_class is None:
-> 2040     raise ValueError(
2041         "Format {!r} is not supported (supported
formats: {})".format(
2042             fmt, ",
".join(sorted(self.get_supported_filetypes()))))
2043 canvas = canvas_class(self.figure)
2044 canvas._is_saving = self._is_saving

```

ValueError: Format '283185307179586, varying alpha' is not supported (supported formats: eps, jpeg, jpg, pdf, pgf, png, ps, raw, rgba, svg, svgz, tif, tiff, webp)



The error occurs because your filename passed to `plt.savefig()` contains characters (like `=`, `,`, `.`, and spaces) that confuse Matplotlib's file format parser. Specifically, this line:

python

Copy code

```
plt.savefig(f"Fixed w_0={omega0}, varying alpha")
```

creates a filename such as:

sql

Copy code

```
Fixed w_0=3.141592653589793, varying alpha
```

Matplotlib interprets the part after the last `.` as a file format — since there's no extension, it tries to read `283185307179586, varying alpha` as the format, which causes the error.

✅ How to Fix It

Add a valid file extension (like `.png`, `.pdf`, or `.svg`) and sanitize the filename:

python

 Copy code

```
plt.savefig(f"Fixed_w0_{omega0:.2f}_varying_alpha.png")
```

This ensures:

- You have a valid image format (.png).
- Special characters and spaces are removed.
- The filename is safe and readable.

Alternative (Safer Formatting)

You can also use `pathlib` to safely construct filenames:

python

 Copy code

```
from pathlib import Path

filename = Path(f"Fixed_w0_{omega0:.2f}_varying_alpha.png")
plt.savefig(filename)
```

In Summary

Replace:

python

 Copy code

```
plt.savefig(f"Fixed w_0={omega0}, varying alpha")
```

with:

python

 Copy code

```
plt.savefig(f"Fixed_w0_{omega0:.2f}_varying_alpha.png")
```

and the error will disappear.

     

